



VNiVERSiDAD
D SALAMANCA

Elaboración de libros electrónicos mediante código y asistentes de Inteligencia Artificial

Curso para las Facultades de Ciencias y de Ciencias Químicas

Álvaro Lozano Murciego y André Sales Mendes

Universidad de Salamanca

ÍNDICE DE CONTENIDOS

1	Sesion 0. Terminal de Ubuntu para Verilog	3
1.1	La terminal como mesa de trabajo	3
1.2	Donde estoy: <code>pwd</code>	3
1.3	Que hay aqui: <code>ls</code>	4
1.4	Cambiar de carpeta: <code>cd</code>	4
1.5	Crear y preparar una carpeta de practicas	4
1.6	Comprobar herramientas instaladas	5
1.7	Crear un primer archivo de texto	5
1.8	Compilar y ejecutar	6
1.9	Patron de trabajo para las sesiones	6
1.10	Comandos basicos de ayuda	7
1.11	Errores frecuentes	7
1.12	Practica guiada	7
1.13	Cierre	8
2	Sesion 1. Introduccion a Verilog	9
2.1	Conceptos clave	9
2.2	Practica guiada	9
2.3	Indice teoria-practica-figuras	10
2.4	Figuras de referencia	10
2.5	Cierre	11
2.6	Fuente original	11
3	Sesion 2. Operaciones con bits	13
3.1	Conceptos clave	13
3.2	Practica guiada	13
3.3	Indice teoria-practica-figuras	14
3.4	Trabajo sin figura asociada	14
3.5	Cierre	14
3.6	Fuente original	14
4	Sesion 3. Puertas logicas	15
4.1	Conceptos clave	15
4.2	Practica guiada	15
4.3	Indice teoria-practica-figuras	16
4.4	Figuras de referencia	16
4.5	Cierre	18
4.6	Fuente original	18
5	Sesion 4. Modulos	23

5.1	Conceptos clave	23
5.2	Practica guiada	23
5.3	Indice teoria-practica-figuras	24
5.4	Figuras de referencia	24
5.5	Cierre	27
5.6	Fuente original	27
6	Sesion 5. Encaminadores y sumadores	29
6.1	Conceptos clave	29
6.2	Practica guiada	29
6.3	Indice teoria-practica-figuras	30
6.4	Figuras de referencia	30
6.5	Cierre	35
6.6	Fuente original	35
7	Sesion 6. Biestables	37
7.1	Conceptos clave	37
7.2	Practica guiada	37
7.3	Indice teoria-practica-figuras	38
7.4	Figuras de referencia	38
7.5	Cierre	41
7.6	Fuente original	41
8	Sesion 7. Registros	43
8.1	Conceptos clave	43
8.2	Practica guiada	43
8.3	Indice teoria-practica-figuras	44
8.4	Figuras de referencia	44
8.5	Cierre	46
8.6	Fuente original	46
9	Sesion 8. Contadores	47
9.1	Conceptos clave	47
9.2	Practica guiada	47
9.3	Indice teoria-practica-figuras	48
9.4	Figuras de referencia	48
9.5	Cierre	52
9.6	Fuente original	53
10	Sesion 9. ALU	55
10.1	Conceptos clave	55
10.2	Practica guiada	55
10.3	Indice teoria-practica-figuras	56
10.4	Figuras de referencia	56
10.5	Cierre	58
10.6	Fuente original	58

Bienvenido al libro de prácticas de **Computadores I**.

Este material acompaña el aprendizaje de los fundamentos de diseño digital usando **Verilog** como lenguaje de descripción de hardware. La asignatura parte de una idea sencilla: un computador no es una caja mágica, sino una composición ordenada de señales, puertas lógicas, módulos, registros, contadores y unidades aritmético-lógicas.

Qué vas a aprender

Al trabajar con este libro aprenderás a:

- describir circuitos digitales mediante módulos Verilog;
- simular el comportamiento de puertas lógicas, buses, biestables, registros y contadores;
- interpretar señales, estados y retardos de propagación;
- construir componentes combinacionales y secuenciales de forma incremental;
- relacionar cada ejercicio con los esquemas y tablas de referencia de la sesión.

Cómo está organizado el curso

El curso se estructura en sesiones prácticas. Cada sesión incluye una breve explicación teórica, ejercicios guiados, un índice que relaciona teoría, práctica y figuras, y esquemas de referencia para no perder de vista el circuito que se está programando.

Método de trabajo recomendado

Antes de escribir código, identifica las entradas, salidas y señales internas del circuito. Después consulta las figuras de referencia, escribe el módulo Verilog y comprueba la simulación. Si el resultado no coincide con lo esperado, vuelve a la figura y revisa las conexiones.

Recorrido de las sesiones

Tabla1: Mapa inicial de la asignatura

Sesión	Tema	Idea principal
0	Terminal de Ubuntu para Verilog	Carpetas, archivos, editores básicos y primer contacto con iverilog
1	Introducción a Verilog	Entorno de trabajo, módulos básicos y salida por pantalla
2	Operaciones con bits	Máscaras, paridad y reducción de bits
3	Puertas lógicas	AND, OR, NOT, NAND, NOR, XOR, XNOR y funciones combinacionales
4	Módulos	Interfaces, jerarquía, comparadores y codificadores
5	Encaminadores y sumadores	Buses, multiplexores, semisumadores y sumadores
6	Biestables	Memoria elemental, reloj, flancos y entradas asíncronas
7	Registros	Registros SISO/SIPO y depuración con cronogramas
8	Contadores	Cuenta ascendente/descendente y transiciones de estado
9	ALU	Inclusión de ficheros y uso de la ALU 74181

Material disponible

Empieza por la sesión de preparación del curso:

- *Sesión 0. Terminal de Ubuntu para Verilog*

También puedes cambiar al libro en inglés desde el selector de idioma de la barra superior.

SESION 0. TERMINAL DE UBUNTU PARA VERILOG

Esta sesion sirve para perder el miedo a la terminal. Antes de escribir circuitos en Verilog conviene saber moverse por carpetas, localizar archivos y ejecutar las herramientas que compilan y simulan nuestros disenos.

Objetivos de aprendizaje

- Abrir una terminal de Ubuntu y saber en que carpeta estamos.
- Movernos por el sistema de archivos con `pwd`, `ls` y `cd`.
- Crear una carpeta de trabajo y organizar ficheros Verilog.
- Crear un archivo sencillo con `gedit` y mostrarlo desde la terminal.
- Preparar el primer comando de compilacion con `iverilog`.

1.1 La terminal como mesa de trabajo

Una terminal es una ventana donde escribimos ordenes de texto. Para este curso la usaremos como una mesa de trabajo: entramos en una carpeta, miramos que archivos hay, abrimos un editor y ejecutamos comandos sencillos.

En Ubuntu puedes abrirla con `Ctrl + Alt + T`, o buscandola como **Terminal** en el menu de aplicaciones.

1.2 Donde estoy: `pwd`

El comando `pwd` muestra la ruta de la carpeta actual.

```
pwd
```

Una salida posible es:

```
/home/usuario
```

Esta ruta significa: estamos dentro de la carpeta personal del usuario. Si la terminal fuera un navegador de archivos, `pwd` seria mirar la barra de direccion.

1.3 Que hay aqui: `ls`

El comando `ls` lista el contenido de la carpeta actual.

```
ls
```

Variantes utiles:

```
ls -l
ls -a
ls -lh
```

Comando	Para que sirve
<code>ls</code>	Lista archivos y carpetas visibles.
<code>ls -l</code>	Muestra mas detalles: permisos, tamaño y fecha.
<code>ls -a</code>	Incluye archivos ocultos, los que empiezan por punto.
<code>ls -lh</code>	Muestra tamaños mas legibles, por ejemplo 4K o 2M.

1.4 Cambiar de carpeta: `cd`

El comando `cd` cambia la carpeta actual.

```
cd Documentos
pwd
```

Para volver una carpeta hacia atras:

```
cd ..
```

Para ir directamente a tu carpeta personal:

```
cd
```

Para entrar en una ruta concreta:

```
cd ~/verilog/sesion_00
```

El simbolo `~` representa tu carpeta personal. En muchas instalaciones equivale a algo como `/home/usuario`.

1.5 Crear y preparar una carpeta de practicas

Vamos a crear una carpeta para el curso y otra para esta sesion.

```
mkdir -p ~/verilog/sesion_00
cd ~/verilog/sesion_00
pwd
```

El comando `mkdir` crea carpetas. La opcion `-p` permite crear varias carpetas encadenadas si todavia no existen.

1.6 Comprobar herramientas instaladas

Para compilar Verilog en Ubuntu usaremos principalmente:

- `iverilog`: compila el diseño y el banco de pruebas.

Comprueba si están disponibles:

```
iverilog -V
```

Si Ubuntu responde con un mensaje de versión, la herramienta está instalada. Si responde `command not found`, falta instalarla.

Instalación en Ubuntu

Si trabajas en una máquina donde tienes permiso para instalar paquetes, puedes preparar las herramientas con:

```
sudo apt update
sudo apt install iverilog
```

1.7 Crear un primer archivo de texto

Crea el archivo `hola.txt` con `gedit`:

```
gedit hola.txt
```

Otros editores en la terminal

Además de `gedit`, existen editores que se abren dentro de la propia terminal, como `nano`, `pico` o `vim`. En esta sesión usaremos `gedit` porque permite empezar de forma visual y sencilla.

Escribe una sola palabra:

```
hola
```

Guarda el archivo y cierra `gedit`.

Comprueba que el archivo existe:

```
ls -l
```

Muestra su contenido desde la terminal:

```
cat hola.txt
```

La salida esperada es:

```
hola
```

1.8 Compilar y ejecutar

Mas adelante escribiremos archivos Verilog. El ciclo sera muy parecido: estar en la carpeta correcta, listar los archivos, compilar y ejecutar el resultado.

Crea este archivo `saludo.v`:

```
gedit saludo.v
```

Contenido:

```
module saludo;
  initial begin
    $display("Hola desde Verilog");
    $finish;
  end
endmodule
```

Compila el archivo con `iverilog`:

```
iverilog -o saludo saludo.v
```

Este comando lee `saludo.v` y crea un archivo ejecutable llamado `saludo`.

Ejecutalo con:

```
./saludo
```

La salida esperada es:

```
Hola desde Verilog
```

1.9 Patron de trabajo para las sesiones

En este curso repetiremos muchas veces la misma secuencia:

```
cd ~/verilog/sesion_00
ls
iverilog -o simulacion archivo.v
./simulacion
```

Cuando el ejemplo tenga un banco de pruebas, normalmente compilaremos dos archivos:

```
iverilog -o simulacion disenio.v testbench.v
./simulacion
```

1.10 Comandos basicos de ayuda

Comando	Uso habitual
<code>clear</code>	Limpia la pantalla de la terminal.
<code>history</code>	Muestra comandos usados anteriormente.
<code>man ls</code>	Abre el manual de <code>ls</code> . Se sale con <code>q</code> .
<code>cat archivo.txt</code>	Muestra el contenido de un archivo corto.
<code>cp origen.v copia.v</code>	Copia un archivo.
<code>mv viejo.v nuevo.v</code>	Renombra o mueve un archivo.
<code>rm archivo.txt</code>	Borra un archivo. Usalo con cuidado.

1.11 Errores frecuentes

Mensaje o situacion	Que revisar
<code>command not found</code>	La herramienta no esta instalada o el nombre esta mal escrito.
<code>No such file or directory</code>	No estas en la carpeta correcta o el archivo no se llama asi. Usa <code>pwd</code> y <code>ls</code> .
<code>cat hola.txt</code> no muestra nada	El archivo esta vacio o no se guardo en <code>gedit</code> .
<code>./saludo</code> no funciona	Revisa que primero hayas compilado con <code>iverilog -o saludo saludo.v</code> .

1.12 Practica guiada

1. Abre una terminal de Ubuntu.
2. Crea la carpeta `~/verilog/sesion_00`.
3. Entra en esa carpeta y confirma la ruta con `pwd`.
4. Abre `gedit` desde la terminal con `gedit hola.txt`.
5. Escribe `hola`, guarda el archivo y cierra `gedit`.
6. Usa `ls` para comprobar que `hola.txt` aparece en la carpeta.
7. Usa `cat hola.txt` para mostrar su contenido.
8. Cambia el texto por `hola mundo`, guarda de nuevo y vuelve a mostrarlo con `cat hola.txt`.

1.13 Cierre

Antes de pasar a la sesión 1, debes poder responder sin mirar apuntes: donde estoy (`pwd`), que archivos tengo (`ls`), como entro en una carpeta (`cd`), como abro un archivo con `gedit` y como muestro un archivo corto con `cat`.

SESION 1. INTRODUCCION A VERILOG

Esta sesion presenta Verilog como lenguaje de descripcion de hardware y prepara el entorno minimo para compilar y simular ejemplos sencillos. El objetivo no es escribir software convencional, sino describir circuitos y observar su comportamiento.

Objetivos de aprendizaje

- Verilog es un HDL: describe hardware, no solo una secuencia de instrucciones.
- El flujo basico de trabajo es escribir un fichero `.v`, compilarlo y ejecutar la simulacion.
- Los modulos delimitan el diseno; `initial` permite definir una secuencia de pruebas.

2.1 Conceptos clave

- Verilog es un HDL: describe hardware, no solo una secuencia de instrucciones.
- El flujo basico de trabajo es escribir un fichero `.v`, compilarlo y ejecutar la simulacion.
- Los modulos delimitan el diseno; `initial` permite definir una secuencia de pruebas.
- Los registros (`reg`) almacenan valores en la simulacion y los cables (`wire`) conectan senales.
- Los formatos de `$display` ayudan a comprobar valores en binario, octal, decimal o hexadecimal.

2.2 Practica guiada

1. Crea un directorio de trabajo y guarda en el un fichero `hello.v`.
2. Compila el fichero con `iverilog` y ejecuta la salida generada.
3. Modifica el ejemplo para imprimir un entero en decimal, binario y hexadecimal; compara tu salida con la [Fig. 2.1](#).
4. Declara un registro de 16 bits y observa que ocurre al asignar valores mayores que su capacidad; usa la [Fig. 2.2](#) para localizar que parte del codigo produce cada salida.

2.3 Indice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el codigo.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Formato de salida y conversion de bases	Ejercicios 3 y 4	Fig. 2.1 , Fig. 2.2

2.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

La [Fig. 2.1](#) sirve como referencia para: ejemplo de salida esperada para una practica inicial de conversion y visualizacion de datos.

```
module ej_1_9;

integer i;
real f;

initial
begin
    i=4;
    f=2.7172;
    $display("i vale %d y f vale %g",i,f);
end

endmodule
```

Figura2.1: Ejemplo de salida esperada para una practica inicial de conversion y visualizacion de datos.

La [Fig. 2.2](#) sirve como referencia para: version comentada del mismo ejemplo, util para localizar cada instruccion en el codigo.

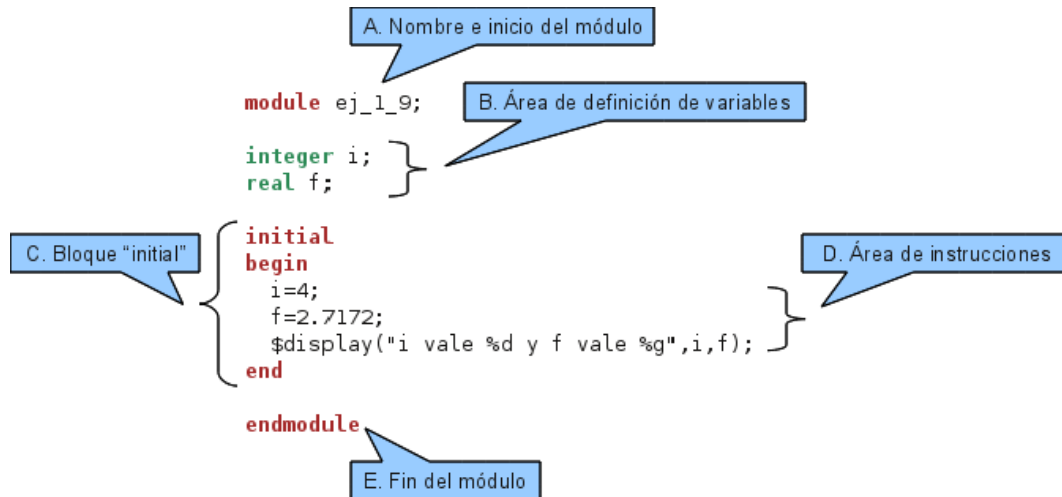


Figura2.2: Version comentada del mismo ejemplo, útil para localizar cada instrucción en el código.

2.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

2.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion1.htm>.

SESION 2. OPERACIONES CON BITS

Esta sesion se centra en el manejo de bits mediante mascarar. La idea central es cambiar solo algunas posiciones de un registro mientras el resto permanece igual.

Objetivos de aprendizaje

- El complemento a uno se obtiene con $\sim$ y cambia cada 0 por 1 y cada 1 por 0.
- Para activar bits se usa una mascara con unos en las posiciones objetivo y una operacion OR bit a bit.
- Para desactivar bits se combina una mascara inversa con una operacion AND bit a bit.

3.1 Conceptos clave

- El complemento a uno se obtiene con $\sim$ y cambia cada 0 por 1 y cada 1 por 0.
- Para activar bits se usa una mascara con unos en las posiciones objetivo y una operacion OR bit a bit.
- Para desactivar bits se combina una mascara inversa con una operacion AND bit a bit.
- Para voltear bits se usa XOR bit a bit con una mascara que marca las posiciones que cambian.
- Las reducciones permiten condensar muchos bits en un unico resultado, por ejemplo para paridad.

3.2 Practica guiada

1. Declara un registro de 16 bits y asigne un valor inicial facil de reconocer en binario.
2. Activa un bit, desactiva tres bits y voltea otro bit usando mascarar.
3. Calcula el bit de paridad par de un registro de 8 bits.
4. Resuelve primero a mano y despues compara con la simulacion; en esta sesion la referencia principal es la tabla de bits que construyas durante el calculo.

3.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios cionados	rela-	Figuras que se deben consultar
Mascaras, paridad y reduccion de bits	Ejercicios 1 a 4		Tabla manual de bits del enunciado; no hay figura externa asociada

3.4 Trabajo sin figura asociada

Esta sesion se apoya sobre todo en operaciones sobre registros y mascaras. Usa una tabla escrita a mano para seguir cada bit antes de ejecutar el código.

3.5 Cierre

Antes de pasar a la sesion siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimio realmente la simulacion. Esa comparacion es la forma mas rapida de localizar errores.

3.6 Fuente original

Contenido redactado y ampliado a partir de la presentacion de clase y de la pagina de referencia: <http://avellano.fis.usal.es/~compi/sesion2.htm>.

SESION 3. PUERTAS LOGICAS

La sesion conecta las puertas logicas vistas en teoria con su instanciacion en Verilog. Tambien introduce los valores especiales x y z , importantes en simulacion digital.

Objetivos de aprendizaje

- Las puertas primitivas de Verilog se instancian indicando salida y entradas.
- Los valores x e z permiten representar incertidumbre y alta impedancia.
- Una tabla de verdad completa debe contemplar 0, 1, x y z cuando el ejercicio lo exige.

4.1 Conceptos clave

- Las puertas primitivas de Verilog se instancian indicando salida y entradas.
- Los valores x e z permiten representar incertidumbre y alta impedancia.
- Una tabla de verdad completa debe contemplar 0, 1, x y z cuando el ejercicio lo exige.
- La interconexion de puertas permite construir funciones combinacionales mas complejas.
- Una misma funcion puede implementarse de varias formas, por ejemplo solo con puertas NAND.

4.2 Practica guiada

1. Completa la tabla de verdad de AND con las 16 combinaciones de 0, 1, x y z ; usa la Fig. 4.1 y la Fig. 4.2 como referencia.
2. Repite la comprobacion para OR, NAND, NOR, XOR, XNOR y BUFFER; consulta de la Fig. 4.3 a la Fig. 4.9 segun la puerta que estes simulando.
3. Construye la funcion propuesta con puertas y verifica su tabla de verdad; usa la Fig. 4.10, la Fig. 4.11 y la Fig. 4.12.
4. Reimplementa la funcion usando solo NAND y compara las salidas; toma como guia la Fig. 4.14, la Fig. 4.13 y la Fig. 4.15.

4.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria		Ejercicios relacionados	Figuras que se deben consultar
Puertas basicas		Ejercicios 1 y 2	Fig. 4.1, Fig. 4.2, Fig. 4.3, Fig. 4.4, Fig. 4.5, Fig. 4.6, Fig. 4.7, Fig. 4.8, Fig. 4.9
Funcion combinatorial f2		Ejercicio 3	Fig. 4.10, Fig. 4.11, Fig. 4.12
Equivalencia NAND	con	Ejercicio 4	Fig. 4.13, Fig. 4.14, Fig. 4.15

4.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

La Fig. 4.1 sirve como referencia para: puerta AND basica.

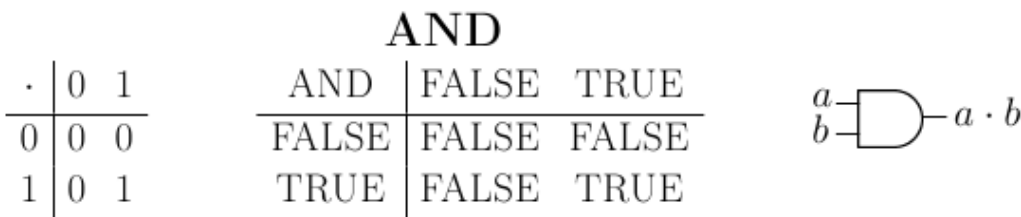


Figura4.1: Puerta AND basica.

La Fig. 4.2 sirve como referencia para: instanciacion de una puerta AND en Verilog.

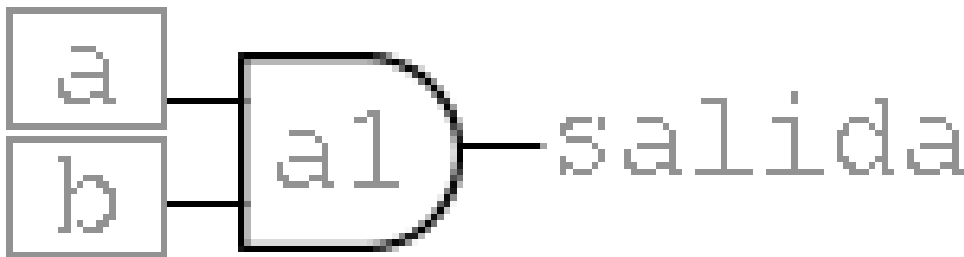


Figura4.2: Instanciacion de una puerta AND en Verilog.

La Fig. 4.3 sirve como referencia para: puerta OR y su conexion conceptual.

La Fig. 4.4 sirve como referencia para: puerta NOT.

La Fig. 4.5 sirve como referencia para: puerta NAND.

La Fig. 4.6 sirve como referencia para: puerta NOR.

OR			
+	0	1	
0	0	1	
1	1	1	

OR	FALSE	TRUE
FALSE	FALSE	TRUE
TRUE	TRUE	TRUE

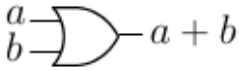


Figura4.3: Puerta OR y su conexion conceptual.

NOT			
-	0	1	
0	1	0	

NOT	FALSE	TRUE
FALSE	TRUE	FALSE
TRUE	FALSE	TRUE

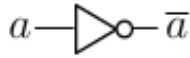


Figura4.4: Puerta NOT.

NAND			
a	b	NAND	
0	0	1	
0	1	1	
1	0	1	
1	1	0	

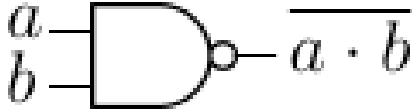


Figura4.5: Puerta NAND.

NOR

a	b	NOR
0	0	1
0	1	0
1	0	0
1	1	0

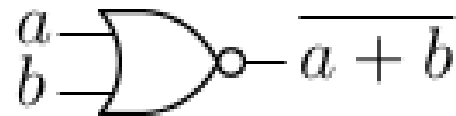


Figura4.6: Puerta NOR.

La Fig. 4.7 sirve como referencia para: puerta XOR.

La Fig. 4.8 sirve como referencia para: puerta XNOR.

La Fig. 4.9 sirve como referencia para: puerta BUFFER.

La Fig. 4.10 sirve como referencia para: funcion logica combinacional f2.

La Fig. 4.11 sirve como referencia para: variables auxiliares de la funcion f2.

La Fig. 4.12 sirve como referencia para: tabla de verdad asociada a f2.

La Fig. 4.13 sirve como referencia para: forma algebraica de la funcion f3.

La Fig. 4.14 sirve como referencia para: implementacion con puertas de f3.

La Fig. 4.15 sirve como referencia para: implementacion equivalente usando NAND.

4.5 Cierre

Antes de pasar a la sesion siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimio realmente la simulacion. Esa comparacion es la forma mas rapida de localizar errores.

4.6 Fuente original

Contenido redactado y ampliado a partir de la presentacion de clase y de la pagina de referencia: <http://avellano.fis.usal.es/~compi/sesion3.htm>.

XOR

a	b	XOR
0	0	0
0	1	1
1	0	1
1	1	0

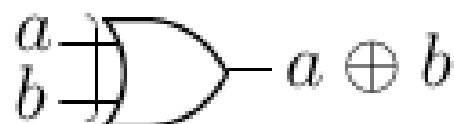


Figura4.7: Puerta XOR.

XNOR

a	b	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

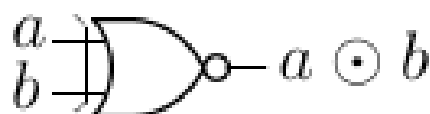


Figura4.8: Puerta XNOR.

BUFFER

0	1
0	1

BUFFER	FALSE	TRUE
	FALSE	TRUE



Figura4.9: Puerta BUFFER.

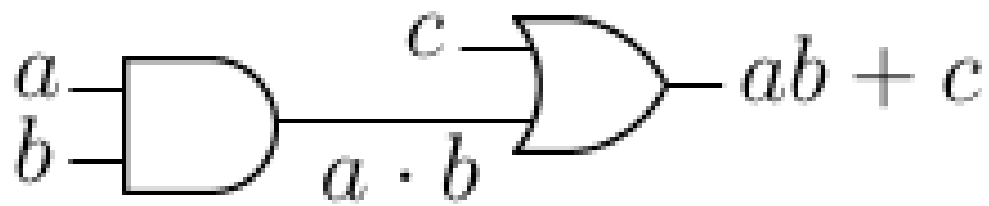


Figura4.10: Funcion logica combinacional f2.

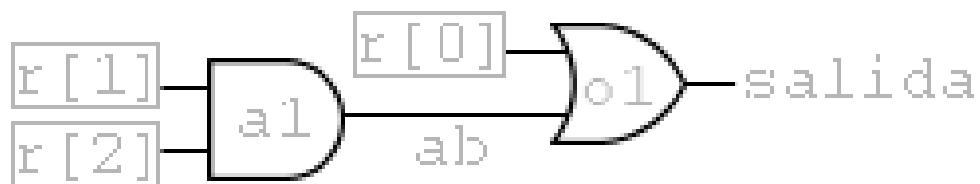


Figura4.11: Variables auxiliares de la funcion f2.

a	b	c	ab	f_2
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Figura4.12: Tabla de verdad asociada a f_2 .

$$f_3(a, b, c) = bc + ab\bar{c} + \bar{b}c + c$$

Figura4.13: Forma algebraica de la funcion f3.

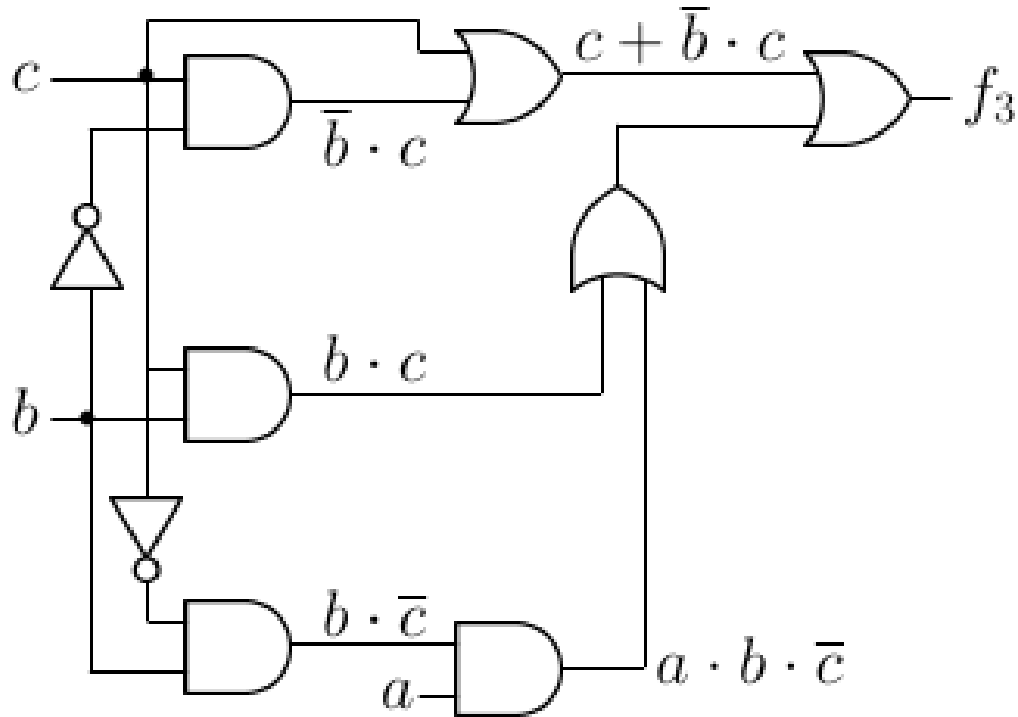


Figura4.14: Implementacion con puertas de f3.

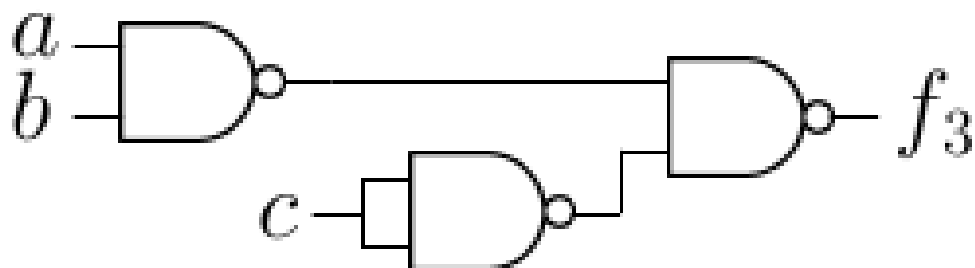


Figura4.15: Implementacion equivalente usando NAND.

SESION 4. MODULOS

Esta sesion introduce el diseno modular. Un modulo en Verilog encapsula entradas, salidas y comportamiento, de forma parecida a una caja reutilizable.

Objetivos de aprendizaje

- Un modulo debe tener una interfaz clara: `input`, `output` e `inout`.
- La jerarquia permite construir sistemas grandes a partir de modulos pequenos.
- Las puertas logicas pueden verse como modulos predefinidos del lenguaje.

5.1 Conceptos clave

- Un modulo debe tener una interfaz clara: `input`, `output` e `inout`.
- La jerarquia permite construir sistemas grandes a partir de modulos pequenos.
- Las puertas logicas pueden verse como modulos predefinidos del lenguaje.
- Un comparador de un bit es un buen primer ejemplo de modulo reutilizable.
- Los codificadores muestran como cambia el circuito cuando se introduce prioridad.

5.2 Practica guiada

1. Define la interfaz de un comparador de un bit; identifica entradas y salidas en la Fig. 5.2.
2. Implementa el comparador con puertas NOT, AND y NOR; sigue la estructura interna de la Fig. 5.3.
3. Crea un modulo de prueba que recorra todas las entradas; compara tu planteamiento con la Fig. 5.4.
4. Usa dos comparadores de un bit para construir un comparador de dos bits; toma como referencia la jerarquia de la Fig. 5.5.

5.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Interfaz de modulo	Ejercicio 1	Fig. 5.1, Fig. 5.2
Circuito interno del comparador	Ejercicio 2	Fig. 5.3
Modulo de comprobacion	Ejercicio 3	Fig. 5.4
Jerarquia y codificadores	Ejercicio 4 y ampliaciones	Fig. 5.5, Fig. 5.6, Fig. 5.7

5.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

La Fig. 5.1 sirve como referencia para: esquema general de un modulo con puertos.

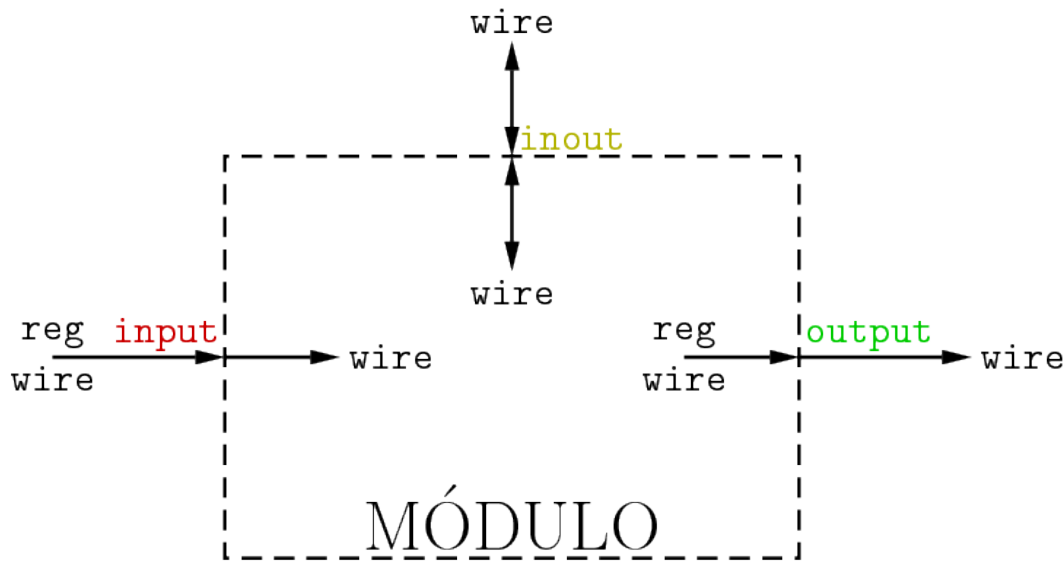


Figura5.1: Esquema general de un modulo con puertos.

La Fig. 5.2 sirve como referencia para: comparador de un bit.

La Fig. 5.3 sirve como referencia para: implementacion interna del comparador de un bit.

La Fig. 5.4 sirve como referencia para: modulo de comprobacion del comparador.

La Fig. 5.5 sirve como referencia para: comparador de dos bits construido jerarquicamente.

La Fig. 5.6 sirve como referencia para: codificador con prioridad.

La Fig. 5.7 sirve como referencia para: puerta AND de cuatro entradas.

a	b	$a > b$	$a = b$	$a < b$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

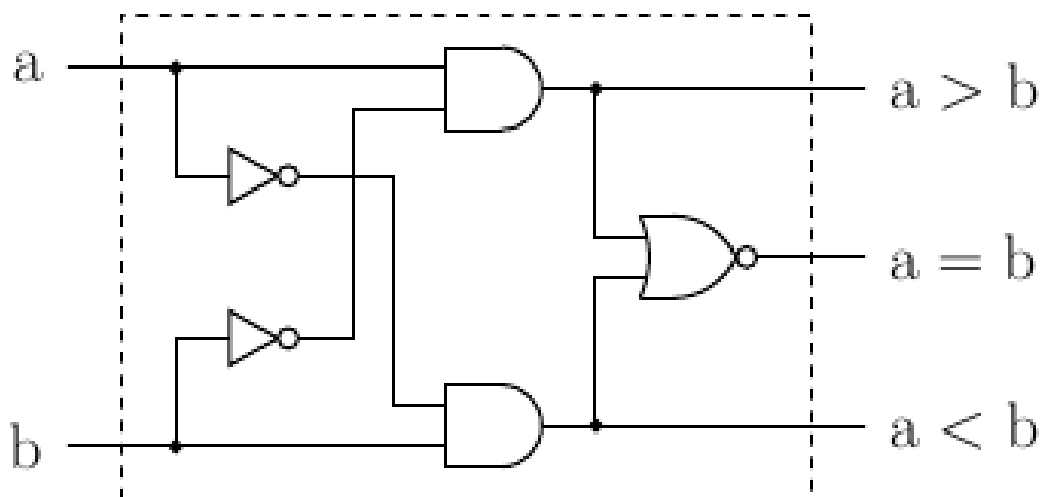


Figura5.2: Comparador de un bit.

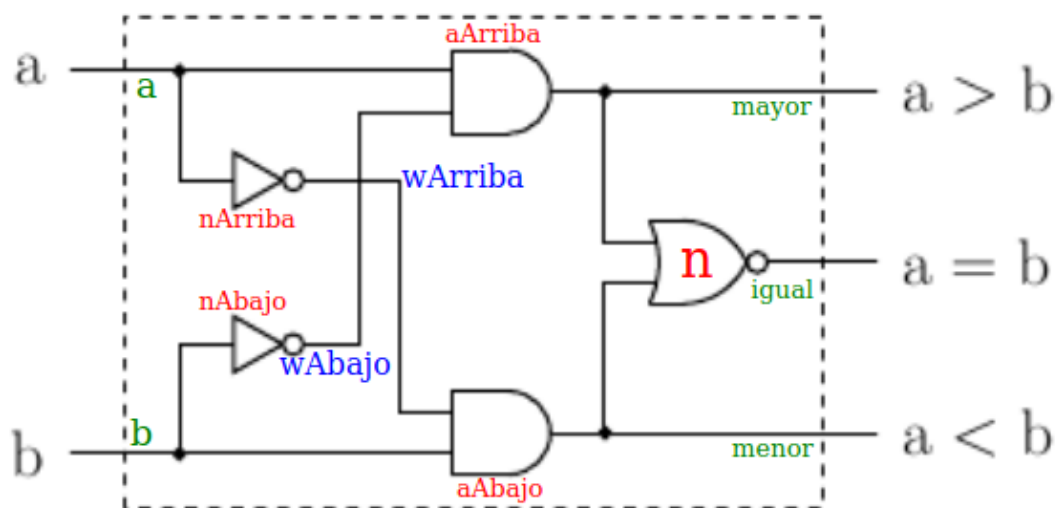


Figura5.3: Implementacion interna del comparador de un bit.

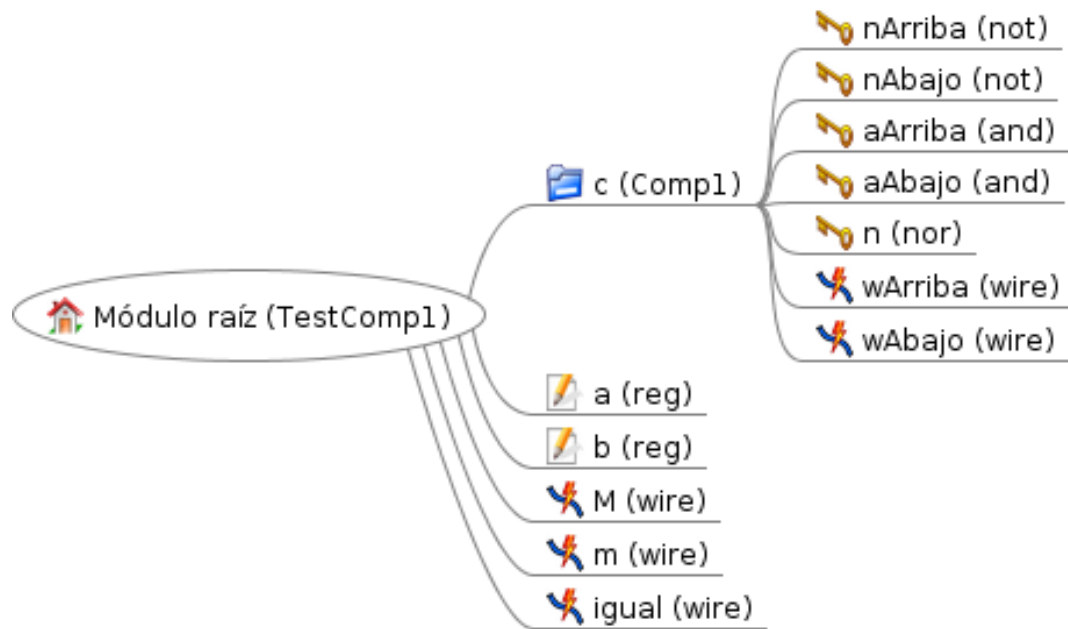


Figura5.4: Módulo de comprobación del comparador.

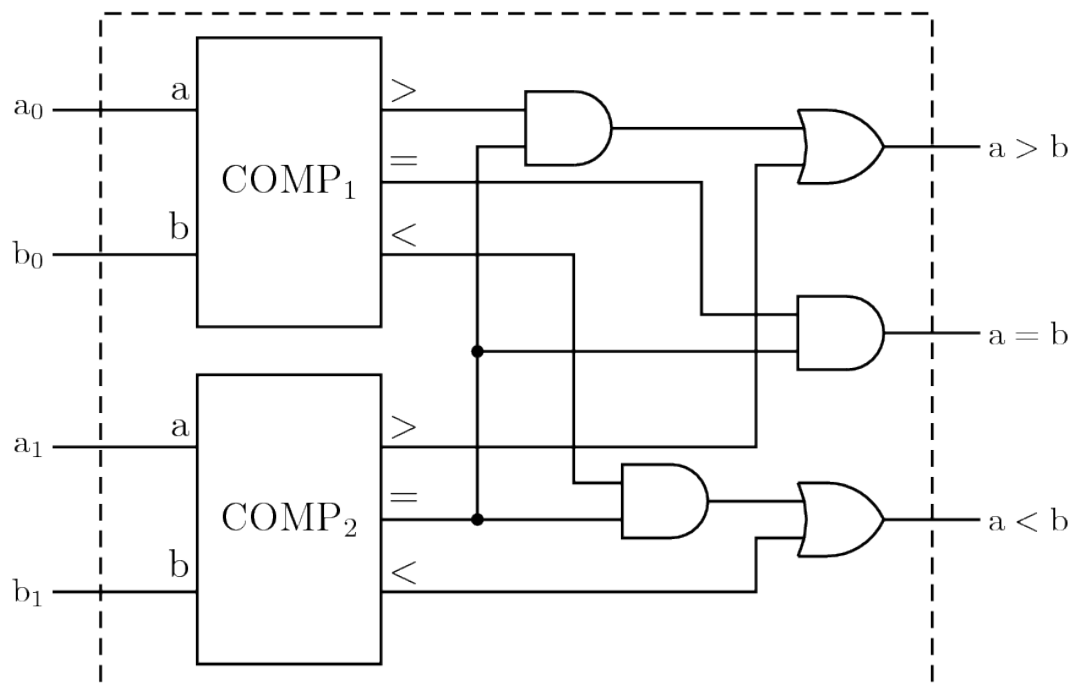


Figura5.5: Comparador de dos bits construido jerárquicamente.

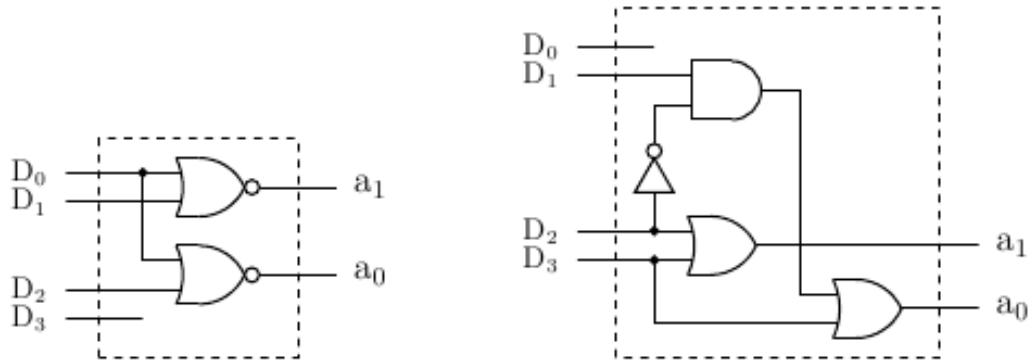


Figura5.6: Codificador con prioridad.

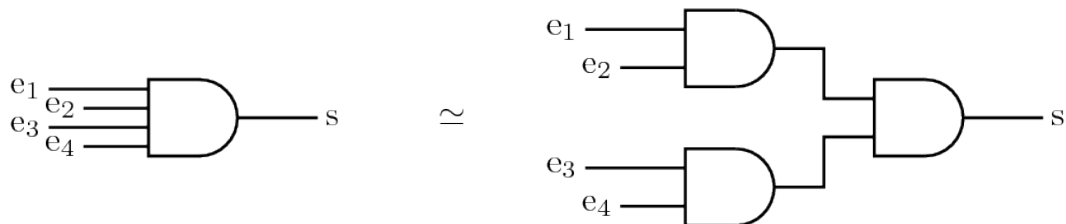


Figura5.7: Puerta AND de cuatro entradas.

5.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

5.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion4.htm>.

SESION 5. ENCAMINADORES Y SUMADORES

La sesion trabaja buses, buffers triestado, multiplexores y sumadores. El hilo conductor es controlar por donde circula una senal y como se propaga el acarreo.

Objetivos de aprendizaje

- Los buffers triestado permiten dejar una salida en alta impedancia.
- Cuando varias senales llegan al mismo cable, conviene usar tipos como `tri`, `tri0`, `tri1`, `wand` o `wor`.
- `assign` crea una conexion continua entre una expresion y un cable.

6.1 Conceptos clave

- Los buffers triestado permiten dejar una salida en alta impedancia.
- Cuando varias senales llegan al mismo cable, conviene usar tipos como `tri`, `tri0`, `tri1`, `wand` o `wor`.
- `assign` crea una conexion continua entre una expresion y un cable.
- Los multiplexores seleccionan una entrada entre varias mediante lineas de control.
- Un sumador completo se puede construir a partir de semisumadores y puertas auxiliares.

6.2 Practica guiada

1. Programa un transmisor/receptor de bus de un bit; usa la [Fig. 6.3](#) y la [Fig. 6.4](#) para nombrar las senales.
2. Construye un multiplexor 4x1 con salida habilitable y despues un 8x1; toma como referencia la [Fig. 6.6](#).
3. Implementa un semisumador y un sumador completo; relaciona tu codigo con la [Fig. 6.8](#) y la [Fig. 6.9](#).
4. Anade retardos y estima el tiempo seguro de estabilizacion en un sumador de cuatro bits; justifica el calculo con la [Fig. 6.10](#) y compara con la alternativa de la [Fig. 6.11](#).

6.3 Índice teoria-practica-figuras

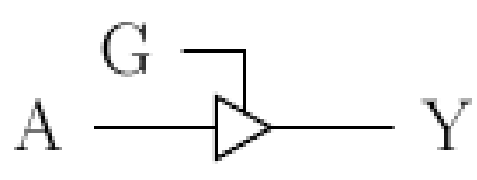
Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Triestado y buses	Ejercicio 1	Fig. 6.1, Fig. 6.2, Fig. 6.3, Fig. 6.4, Fig. 6.5
Multiplexores	Ejercicio 2	Fig. 6.6, Fig. 6.7
Sumadores	Ejercicio 3	Fig. 6.8, Fig. 6.9
Retardos y acarreo	Ejercicio 4	Fig. 6.10, Fig. 6.11

6.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

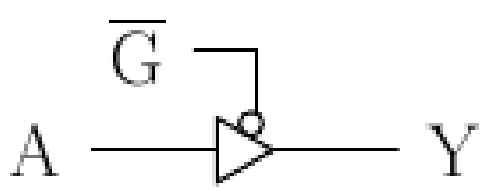
La Fig. 6.1 sirve como referencia para: buffer triestado activo a nivel alto.



G	A	Y
0	x	z
1	0	0
1	1	1

Figura6.1: Buffer triestado activo a nivel alto.

La Fig. 6.2 sirve como referencia para: buffer triestado activo a nivel bajo.



$\overline{G}$	A	Y
0	0	0
0	1	1
1	x	z

Figura6.2: Buffer triestado activo a nivel bajo.

La Fig. 6.3 sirve como referencia para: transmisor/receptor de bus de un bit.

La Fig. 6.4 sirve como referencia para: nombres de senales auxiliares en el transceptor.

La Fig. 6.5 sirve como referencia para: modulo de comprobacion del transceptor.

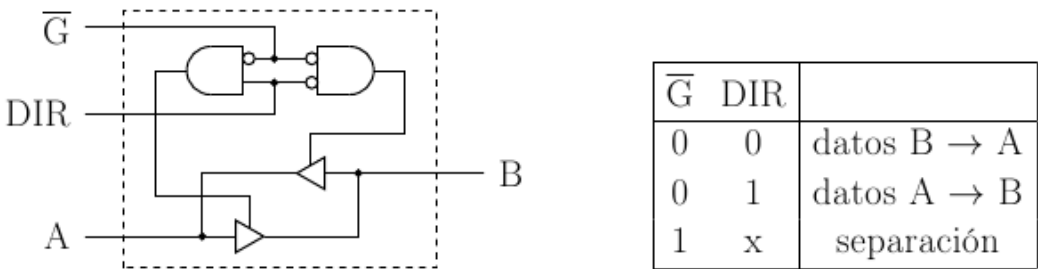


Figura6.3: Transmisor/receptor de bus de un bit.

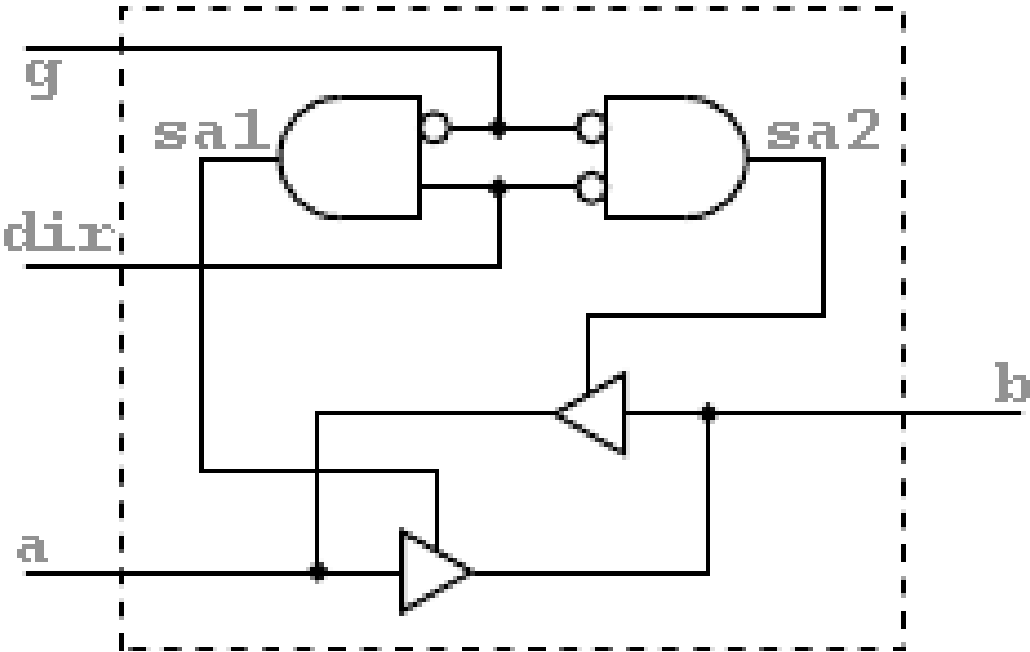


Figura6.4: Nombres de senales auxiliares en el transceptor.

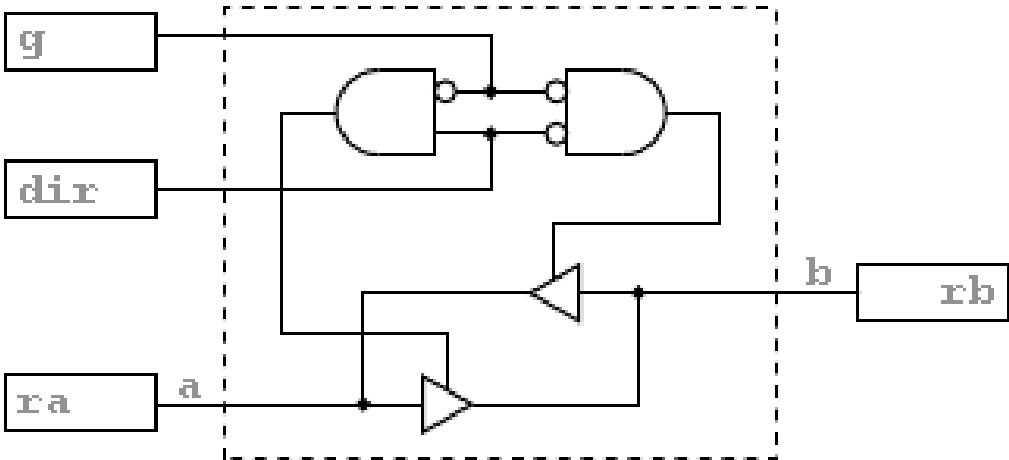


Figura6.5: Modulo de comprobacion del transceptor.

La Fig. 6.6 sirve como referencia para: multiplexor 8x1 construido a partir de multiplexores menores.

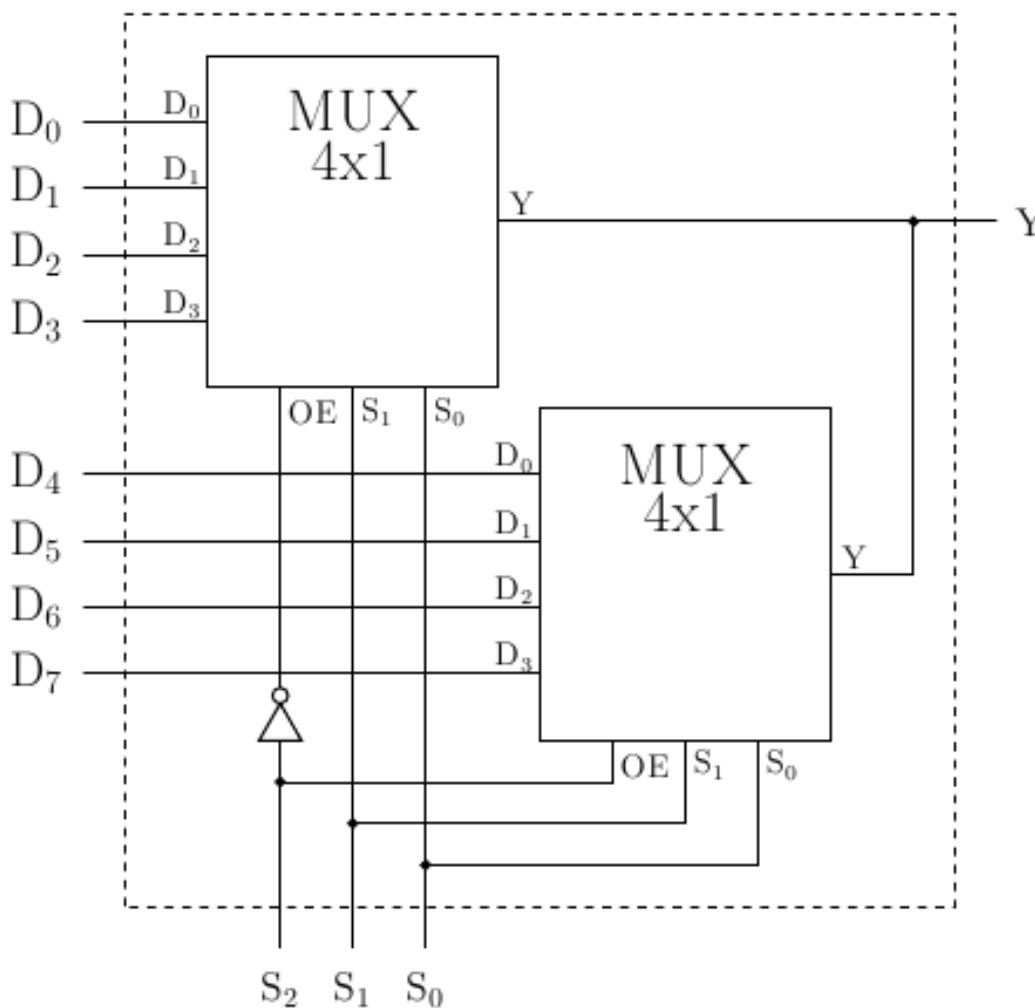


Figura6.6: Multiplexor 8x1 construido a partir de multiplexores menores.

La Fig. 6.7 sirve como referencia para: estructura auxiliar para seleccion de senales.

$$h(a, b, c, d) = \bar{a}\bar{b}\bar{d} + b\bar{c}d + ab + a\bar{c}$$

Figura6.7: Estructura auxiliar para seleccion de senales.

La Fig. 6.8 sirve como referencia para: semisumador.

La Fig. 6.9 sirve como referencia para: sumador completo de un bit.

La Fig. 6.10 sirve como referencia para: sumador de cuatro bits con propagacion de acarreo.

La Fig. 6.11 sirve como referencia para: sumador con anticipacion de acarreo.

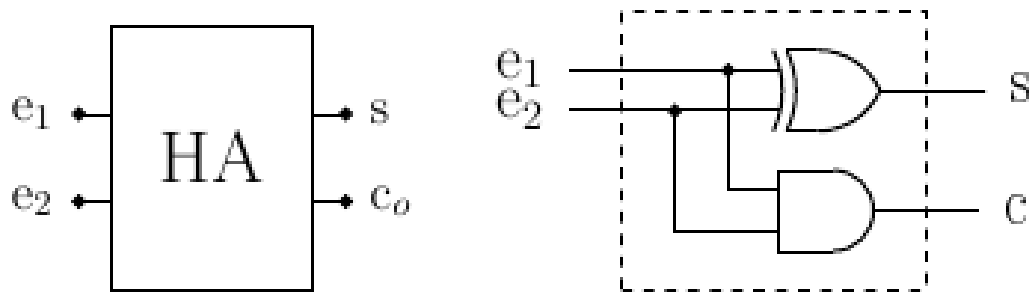


Figura6.8: Semisumador.

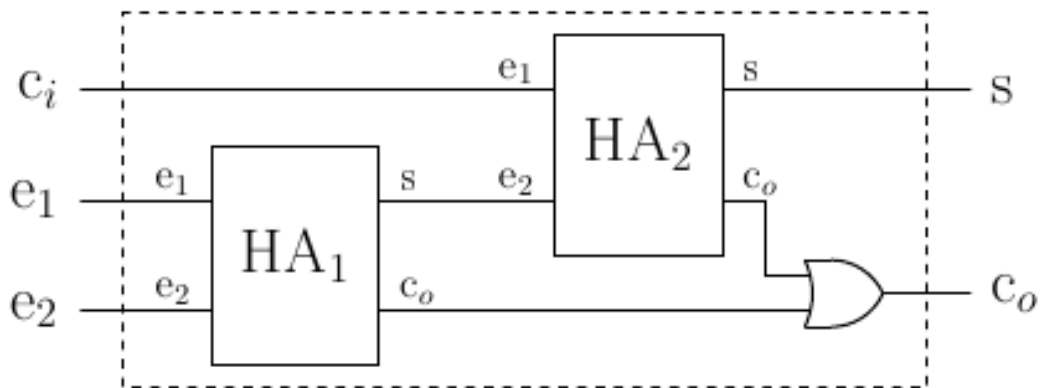


Figura6.9: Sumador completo de un bit.

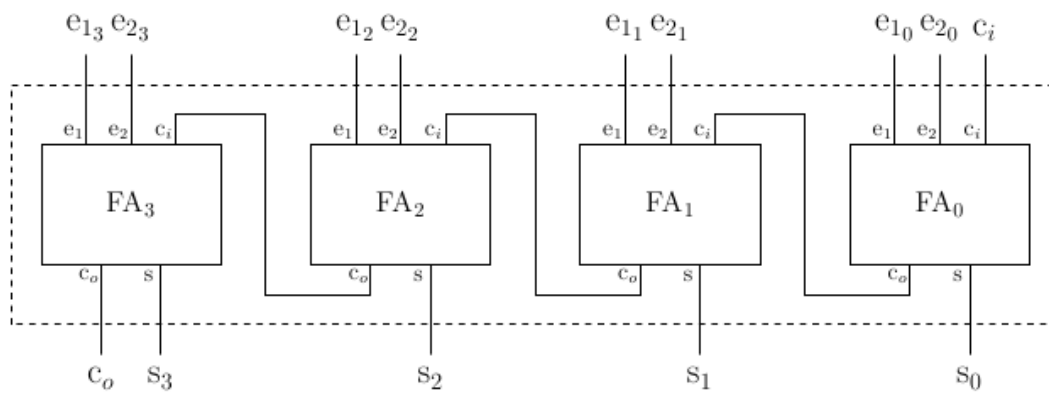


Figura6.10: Sumador de cuatro bits con propagacion de acarreo.

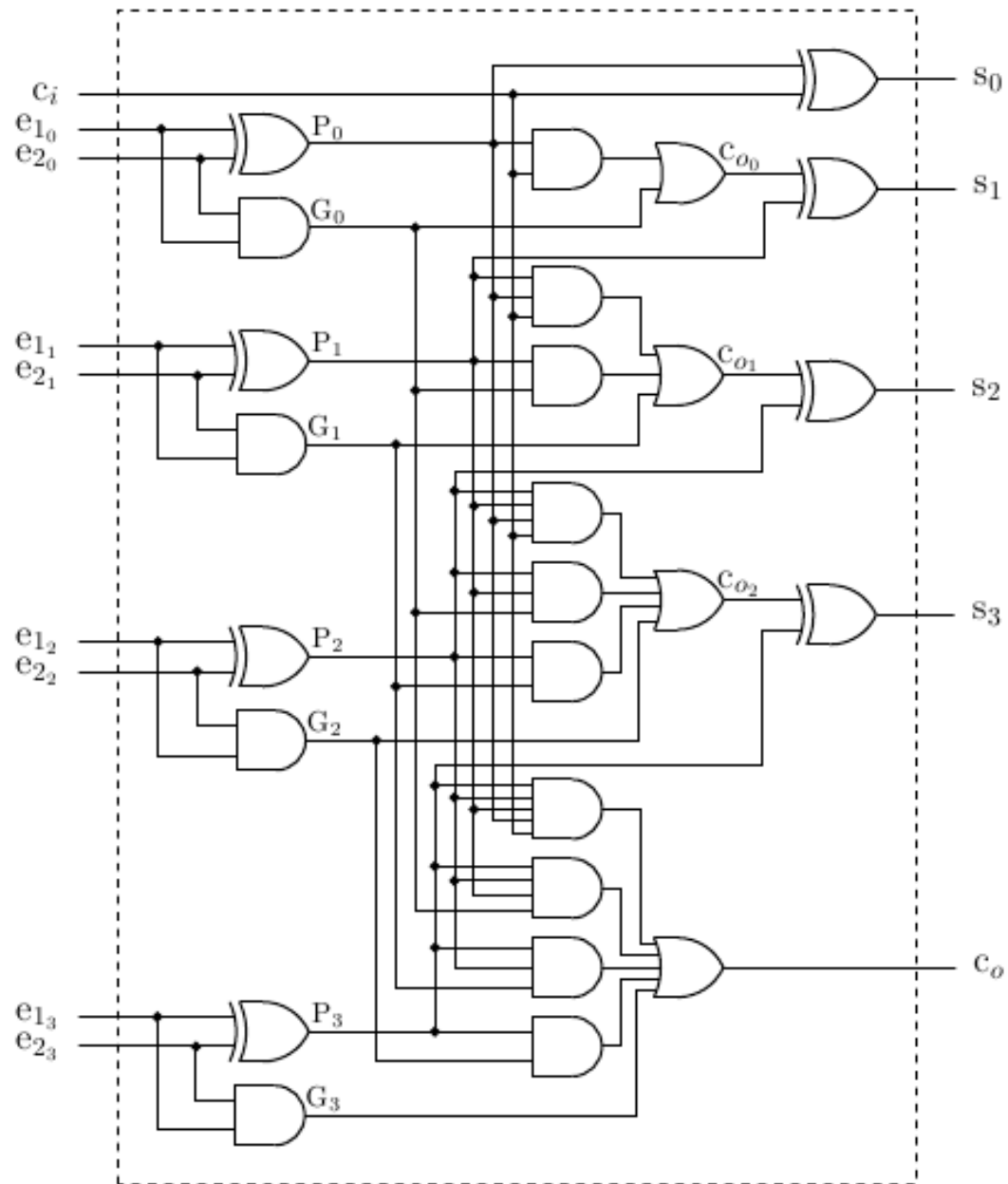


Figura6.11: Sumador con anticipacion de acarreo.

6.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

6.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion5.htm>.

SESION 6. BIESTABLES

Esta sesion pasa de la logica combinacional a la memoria. Los biestables mantienen estado y obligan a pensar en reloj, flancos, entradas asincronas y retardos.

Objetivos de aprendizaje

- Un biestable RS puede construirse con dos puertas NOR realimentadas.
- Las asignaciones bloqueantes y no bloqueantes no producen siempre el mismo efecto temporal.
- Un reloj puede generarse con un bloque `always` que cambia periodicamente una senal.

7.1 Conceptos clave

- Un biestable RS puede construirse con dos puertas NOR realimentadas.
- Las asignaciones bloqueantes y no bloqueantes no producen siempre el mismo efecto temporal.
- Un reloj puede generarse con un bloque `always` que cambia periodicamente una senal.
- Los detectores de flanco convierten una condicion de nivel en un pulso breve.
- PRESET y CLEAR suelen actuar de forma asincrona y tienen prioridad sobre el reloj.

7.2 Practica guiada

1. Programa un biestable RS y prueba la bateria de entradas propuesta; consulta la [Fig. 7.1](#) y contrasta los resultados con la [Fig. 7.2](#).
2. Sustituye algunas asignaciones por no bloqueantes y observa las diferencias cerca de los estados invalidos indicados en la [Fig. 7.2](#).
3. Anade una entrada de reloj activa por nivel; usa la [Fig. 7.3](#) y la [Fig. 7.4](#) para comprobar cuando debe cambiar la salida.
4. Construye una version activa por flanco y despues un biestable JK con PRESET y CLEAR; toma como guia la [Fig. 7.5](#), la [Fig. 7.6](#) y la [Fig. 7.7](#).

7.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Biestable RS	Ejercicios 1 y 2	Fig. 7.1, Fig. 7.2
Reloj por nivel	Ejercicio 3	Fig. 7.3, Fig. 7.4
Flancos y JK	Ejercicio 4	Fig. 7.5, Fig. 7.6, Fig. 7.7

7.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesión.

La Fig. 7.1 sirve como referencia para: biestable RS con puertas NOR.

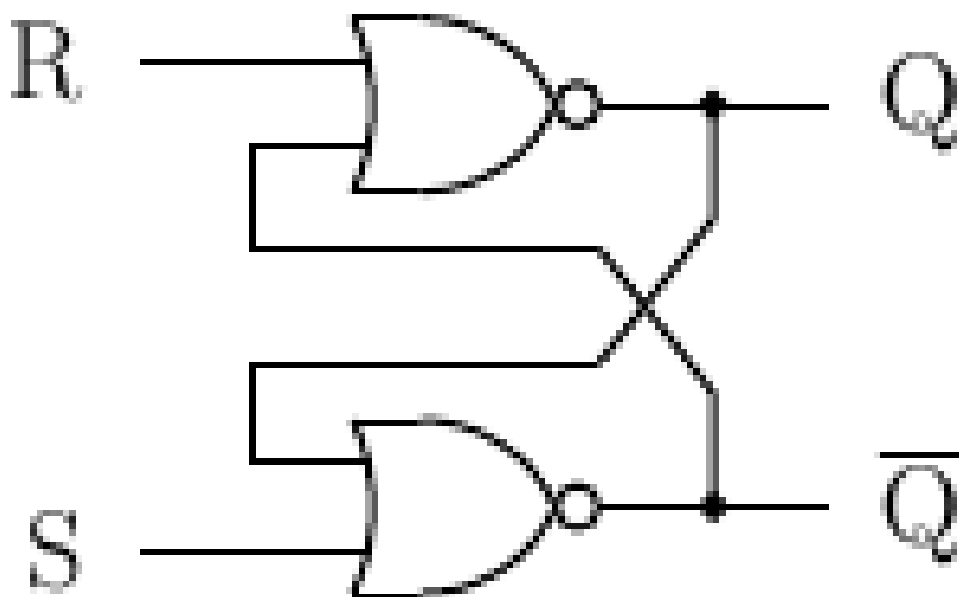


Figura7.1: Biestable RS con puertas NOR.

La Fig. 7.2 sirve como referencia para: tabla de funcionamiento del biestable RS.

La Fig. 7.3 sirve como referencia para: biestable RS con reloj.

La Fig. 7.4 sirve como referencia para: tabla del biestable RS controlado por reloj.

La Fig. 7.5 sirve como referencia para: detector de flanco.

La Fig. 7.6 sirve como referencia para: biestable JK con entradas de control.

La Fig. 7.7 sirve como referencia para: tabla de funcionamiento del biestable JK.

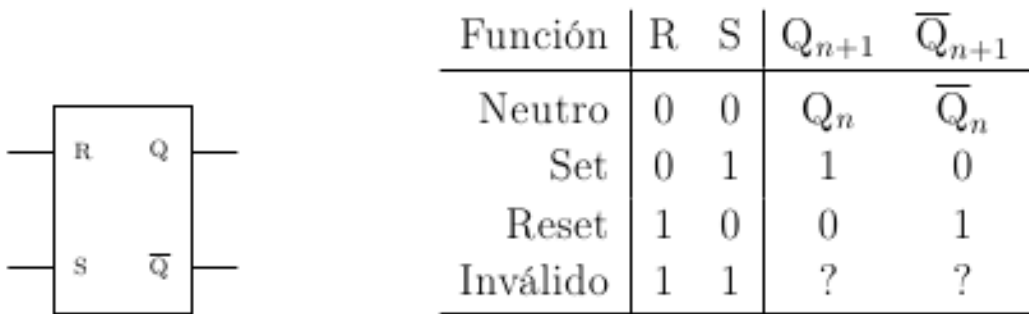


Figura7.2: Tabla de funcionamiento del biestable RS.

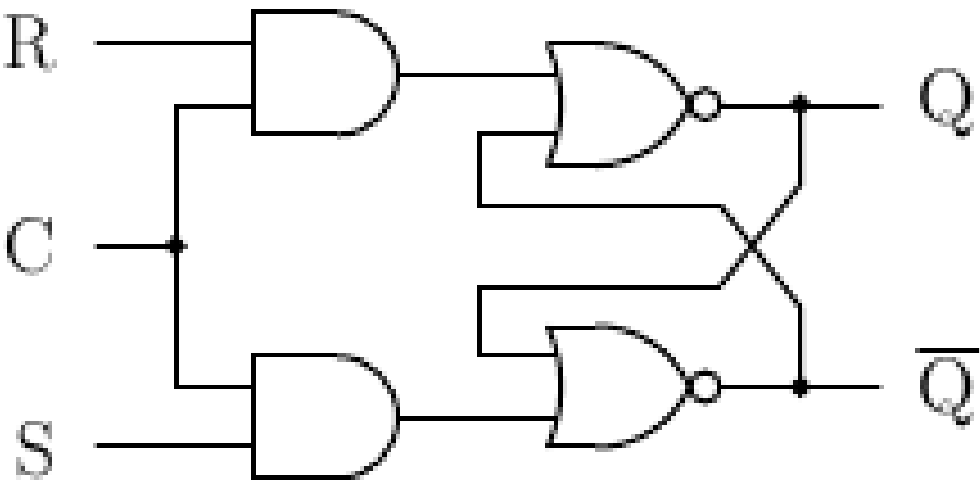


Figura7.3: Biestable RS con reloj.

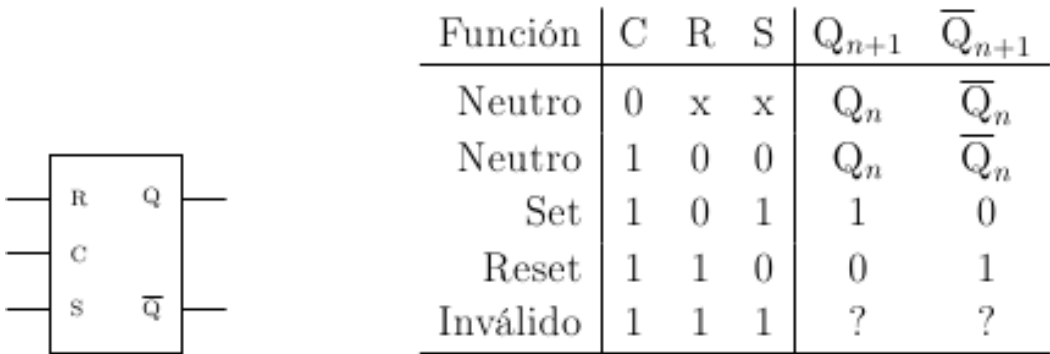


Figura7.4: Tabla del biestable RS controlado por reloj.

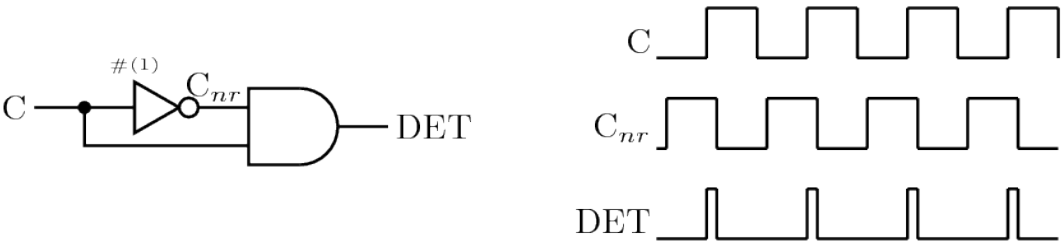


Figura7.5: Detector de flanco.

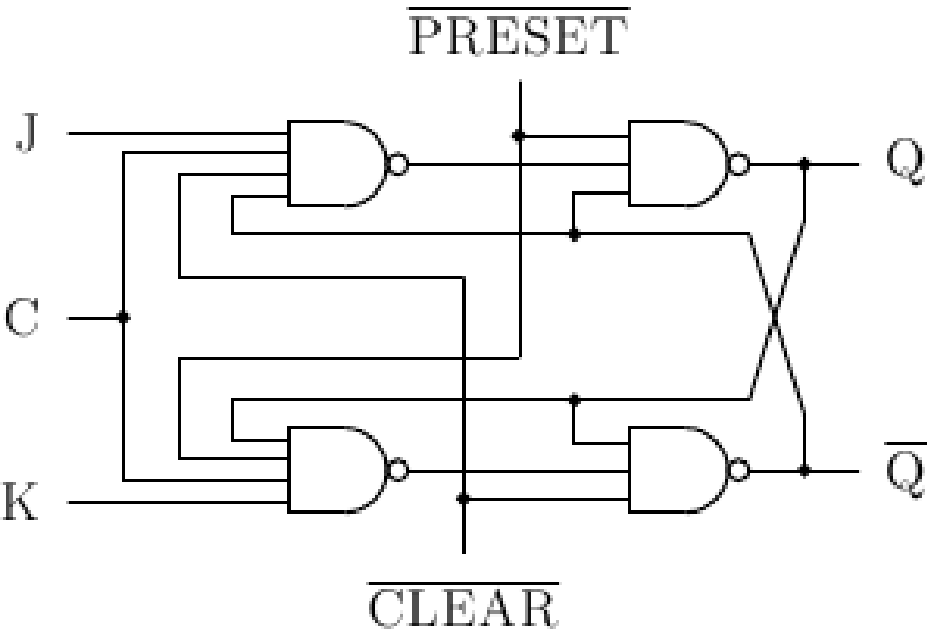


Figura7.6: Biestable JK con entradas de control.

	Función	$\overline{PR}$	$\overline{CL}$	C	J	K	Q_{n+1}	$\overline{Q}_{n+1}$
	Preset	0	1	x	x	x	1	0
	Clear	1	0	x	x	x	0	1
	Inválido	0	0	x	x	x	?	?
	Neutro	1	1	1	0	0	Q_n	$\overline{Q}_n$
	Set	1	1	1	1	0	1	0
	Reset	1	1	1	0	1	0	1
	Complemento	1	1	1	1	1	$\overline{Q}_n$	Q_n
	Neutro	1	1	0	x	x	Q_n	$\overline{Q}_n$

Figura7.7: Tabla de funcionamiento del biestable JK.

7.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

7.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion6.htm>.

SESION 7. REGISTROS

La sesion construye registros a partir de biestables y presenta una herramienta esencial de depuracion: los cronogramas con GTKWave.

Objetivos de aprendizaje

- Un registro almacena varios bits distribuidos en biestables.
- Los registros SISO desplazan informacion serie a serie.
- Los registros SIPO reciben en serie y entregan en paralelo.

8.1 Conceptos clave

- Un registro almacena varios bits distribuidos en biestables.
- Los registros SISO desplazan informacion serie a serie.
- Los registros SIPO reciben en serie y entregan en paralelo.
- `$dumpfile` y `$dumpvars` generan ficheros de ondas para inspeccionar senales.
- Los retardos reales o simulados pueden explicar comportamientos inesperados.

8.2 Practica guiada

1. Construye un biestable D a partir de los bloques ya conocidos; usa la [Fig. 8.1](#) para verificar entradas y salida.
2. Usa varios biestables para formar un registro SISO; sigue la conexion de la [Fig. 8.2](#).
3. Genera un fichero de ondas y abrelo con GTKWave; compara la ventana esperada con la [Fig. 8.3](#).
4. Modifica el registro para obtener una salida paralela SIPO; usa la [Fig. 8.4](#) y, para la ampliacion, la [Fig. 8.5](#).

8.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Biastable D	Ejercicio 1	Fig. 8.1
Registro SISO	Ejercicio 2	Fig. 8.2
Cronogramas	Ejercicio 3	Fig. 8.3
Registros SIPO y carga paralela	Ejercicio 4	Fig. 8.4, Fig. 8.5

8.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

La Fig. 8.1 sirve como referencia para: biastable D.

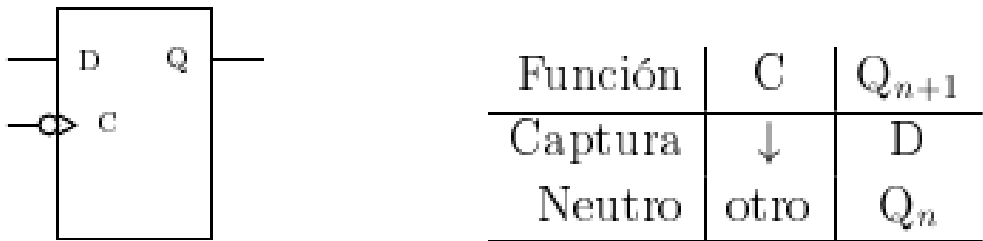


Figura8.1: Biastable D.

La Fig. 8.2 sirve como referencia para: registro SISO.

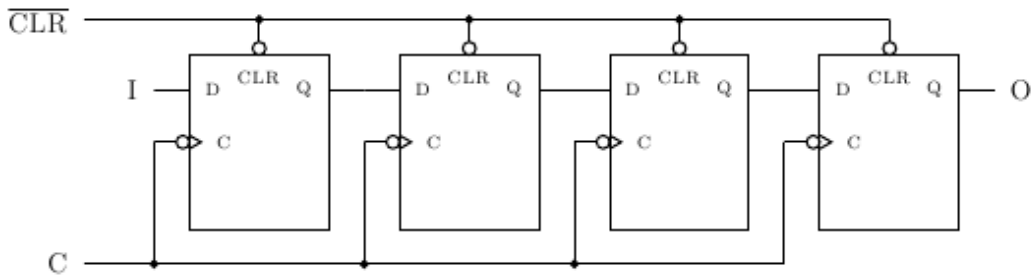


Figura8.2: Registro SISO.

La Fig. 8.3 sirve como referencia para: visualizacion de senales con GTKWave.

La Fig. 8.4 sirve como referencia para: registro SIPO.

La Fig. 8.5 sirve como referencia para: registro con carga paralela y desplazamiento serie.

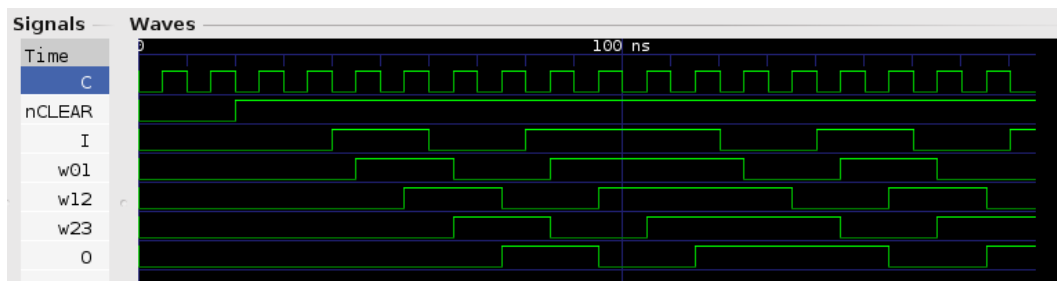


Figura8.3: Visualizacion de senales con GTKWave.

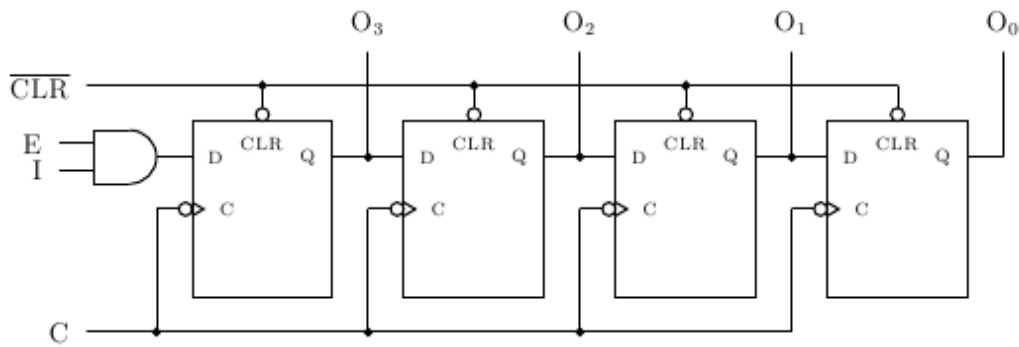


Figura 8.4: Registro SIPO.

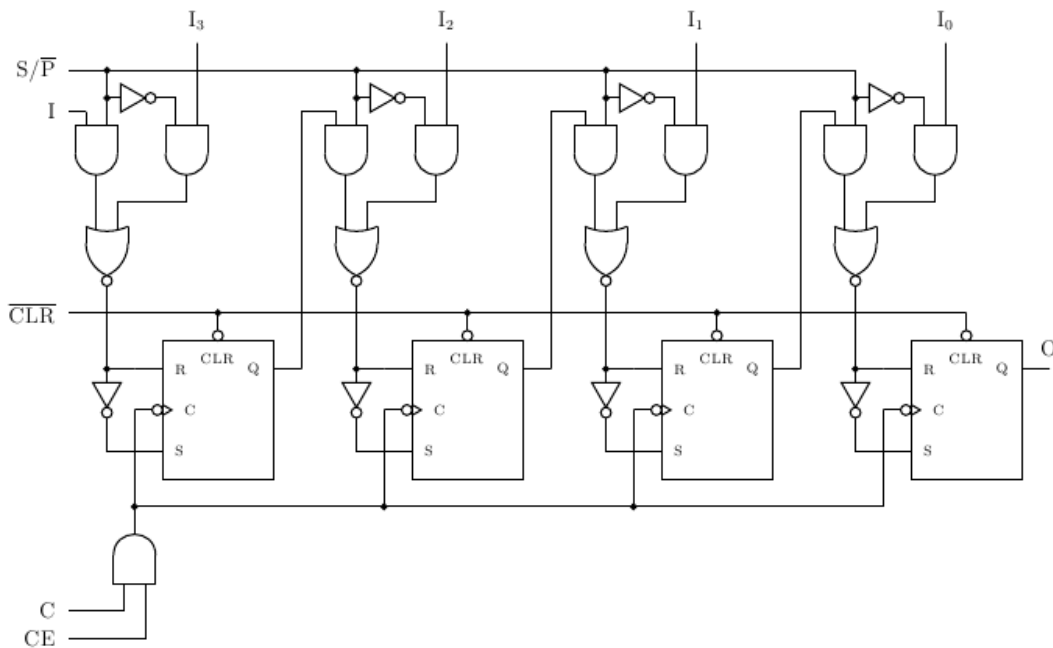


Figura8.5: Registro con carga paralela y desplazamiento serie.

8.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

8.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion7.htm>.

SESION 8. CONTADORES

Esta sesion usa biestables para construir contadores y analizar transiciones de estado. El punto importante es distinguir entre comportamiento ideal, retardos y estados transitorios.

Objetivos de aprendizaje

- Un contador cambia de estado al ritmo de una senal de reloj.
- La cuenta puede ser ascendente o descendente mediante una linea de modo.
- HOLD permite congelar el estado cuando se necesita cambiar de modo sin saltos.

9.1 Conceptos clave

- Un contador cambia de estado al ritmo de una senal de reloj.
- La cuenta puede ser ascendente o descendente mediante una linea de modo.
- HOLD permite congelar el estado cuando se necesita cambiar de modo sin saltos.
- Los contadores asincronos pueden pasar por estados transitorios durante la propagacion.
- Los diagramas de transicion resumen el comportamiento secuencial del circuito.

9.2 Practica guiada

1. Programa un contador ascendente/descendente de cuatro bits; usa la Fig. 9.1 para definir entradas, salidas y modo de cuenta.
2. Observa que ocurre al cambiar de modo sin limpiar el contador; explica los estados transitorios con ayuda de la Fig. 9.4.
3. Anade PRESET, CLEAR y HOLD para controlar mejor las transiciones; compara tu diseno con la Fig. 9.3 y la Fig. 9.5.
4. Analiza un contador de cuenta arbitraria y dibuja su diagrama de estados; usa la Fig. 9.7, la Fig. 9.8 y la Fig. 9.9.

9.3 Indice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Cuenta ascendente/descendente	Ejercicios 1 y 2	Fig. 9.1, Fig. 9.4
Control de transiciones	Ejercicio 3	Fig. 9.2, Fig. 9.3, Fig. 9.5
Estados y biestables JK	Ejercicio 4	Fig. 9.6, Fig. 9.7, Fig. 9.8, Fig. 9.9

9.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesion.

La Fig. 9.1 sirve como referencia para: contador con seleccion de cuenta ascendente o descendente.

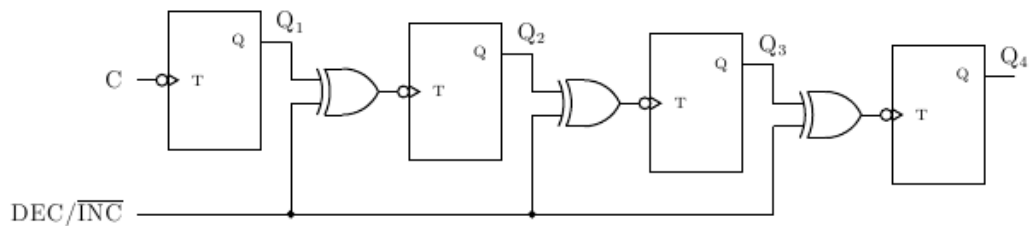


Figura9.1: Contador con seleccion de cuenta ascendente o descendente.

La Fig. 9.2 sirve como referencia para: contador de 10 a 0.

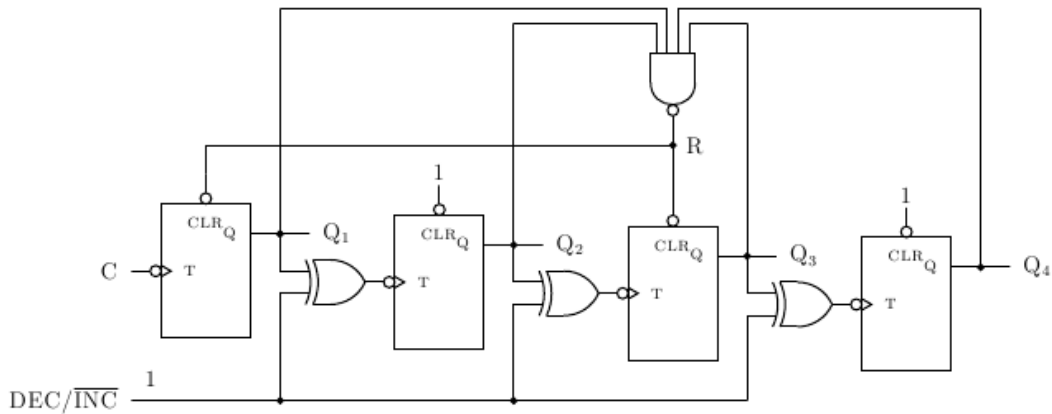


Figura9.2: Contador de 10 a 0.

- La Fig. 9.3 sirve como referencia para: contador sincrono.
- La Fig. 9.4 sirve como referencia para: analisis de estados y transiciones.
- La Fig. 9.5 sirve como referencia para: circuito contador auxiliar.
- La Fig. 9.6 sirve como referencia para: transiciones de un biestable JK.

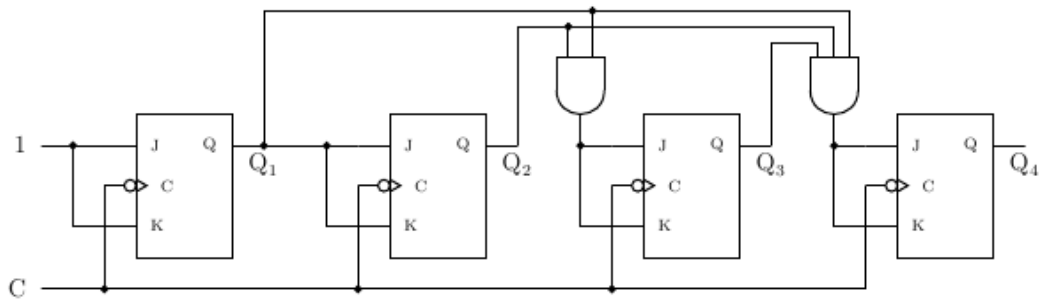


Figura9.3: Contador sincrónico.

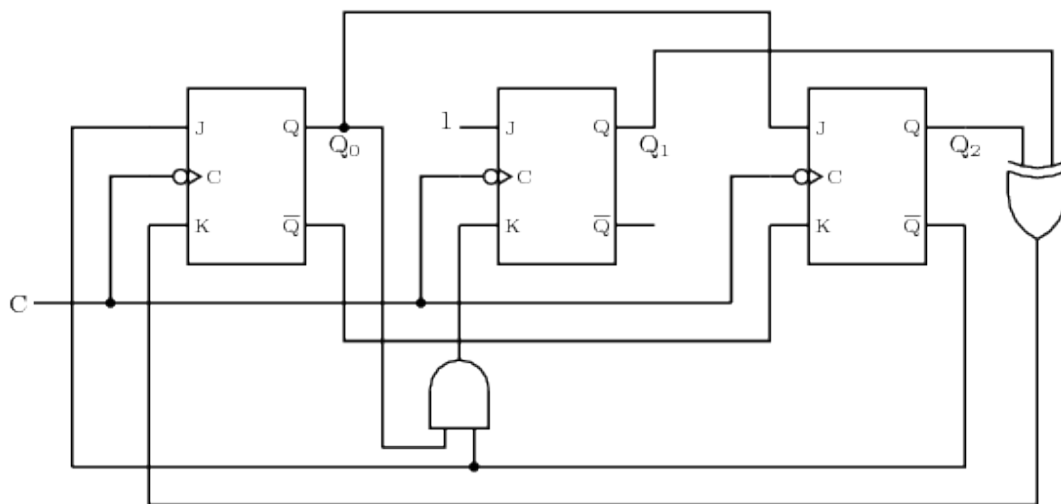


Figura9.4: Analisis de estados y transiciones.

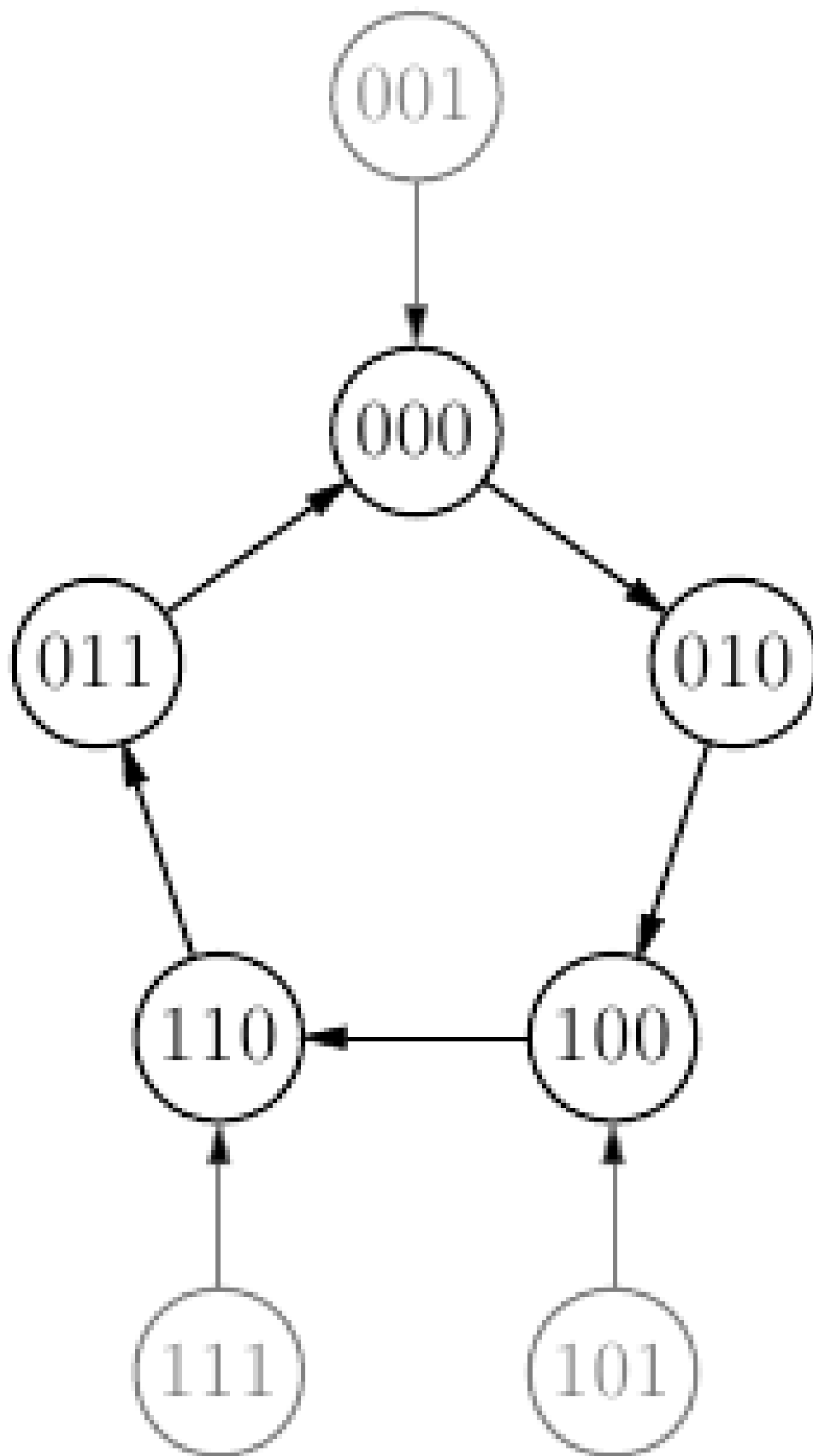


Figura9.5: Circuito contador auxiliar.

Actual	Sgte.	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Figura9.6: Transiciones de un biestable JK.

Transición	J ₂	K ₂	J ₁	K ₁	J ₀	K ₀
000 → 010	0	x	1	x	0	x
001 → 000	0	x	0	x	x	1
010 → 100	1	x	x	1	0	x
011 → 000	0	x	x	1	x	1
100 → 110	x	0	1	x	0	x
101 → 100	x	0	0	x	x	1
110 → 011	x	1	x	0	1	x
111 → 110	x	0	x	0	x	1

Figura9.7: Diagrama de transicion de estados.

La Fig. 9.7 sirve como referencia para: diagrama de transición de estados.

La Fig. 9.8 sirve como referencia para: mapa de Karnaugh para entradas JK.

		J_2		$\leftarrow Q_2$
$Q_1 Q_0 \downarrow$		0	1	
00		0	x	
01		0	x	
11		0	x	
10		1	x	

$$J_2 = Q_1 \overline{Q_0}$$

		K_2		$\leftarrow Q_2$
$Q_1 Q_0 \downarrow$		0	1	
00		x	0	
01		x	0	
11		x	0	
10		x	1	

$$K_2 = Q_1 \overline{Q_0}$$

Figura9.8: Mapa de Karnaugh para entradas JK.

La Fig. 9.9 sirve como referencia para: contador de cuenta arbitraria.

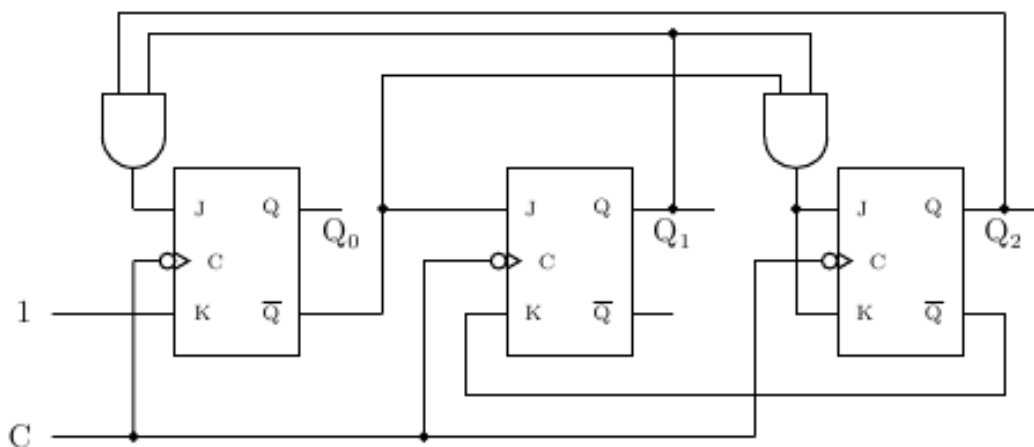


Figura9.9: Contador de cuenta arbitraria.

9.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

9.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion8.htm>.

SESION 9. ALU

La ultima sesion aplica el trabajo modular a una ALU 74181. Se practica la inclusion de ficheros y la combinacion de operaciones aritmeticas y logicas.

Objetivos de aprendizaje

- `include` permite insertar el contenido de otro fichero Verilog en el punto indicado.
- La ALU 74181 recibe lineas de seleccion que determinan la operacion.
- Las lineas de acarreo negadas requieren atencion al interpretar entradas y salidas.

10.1 Conceptos clave

- `include` permite insertar el contenido de otro fichero Verilog en el punto indicado.
- La ALU 74181 recibe lineas de seleccion que determinan la operacion.
- Las lineas de acarreo negadas requieren atencion al interpretar entradas y salidas.
- Las operaciones compuestas se resuelven almacenando resultados intermedios.
- El modulo de prueba debe dejar claras las entradas, la operacion seleccionada y la salida esperada.

10.2 Practica guiada

1. Incluye el fichero `74181.v` desde un modulo de prueba; identifica los puertos en la [Fig. 10.1](#).
2. Comprueba operaciones como $7+4$, $2+6+1$, AND, XOR y OR combinado con NOT/XNOR; selecciona las lineas de control con la [Fig. 10.2](#).
3. Implementa multiplicaciones pequenas como sumas repetidas; usa de nuevo la [Fig. 10.2](#) para justificar cada paso intermedio.
4. Documenta cada seleccion de lineas de control para poder depurar errores; tu tabla debe referirse explicitamente a la [Fig. 10.1](#) y a la [Fig. 10.2](#).

10.3 Índice teoria-practica-figuras

Usa esta tabla como mapa rapido: cuando un ejercicio mencione una figura, esa figura forma parte del enunciado y debe consultarse antes de escribir el código.

Bloque de teoria	Ejercicios relacionados	Figuras que se deben consultar
Interfaz de la ALU	Ejercicio 1	Fig. 10.1
Selección de operaciones	Ejercicios 2 a 4	Fig. 10.2

10.4 Figuras de referencia

Las figuras siguientes recogen los esquemas y tablas que conviene tener a mano mientras se resuelven los ejercicios de la sesión.

La Fig. 10.1 sirve como referencia para: esquema funcional de la ALU 74181.

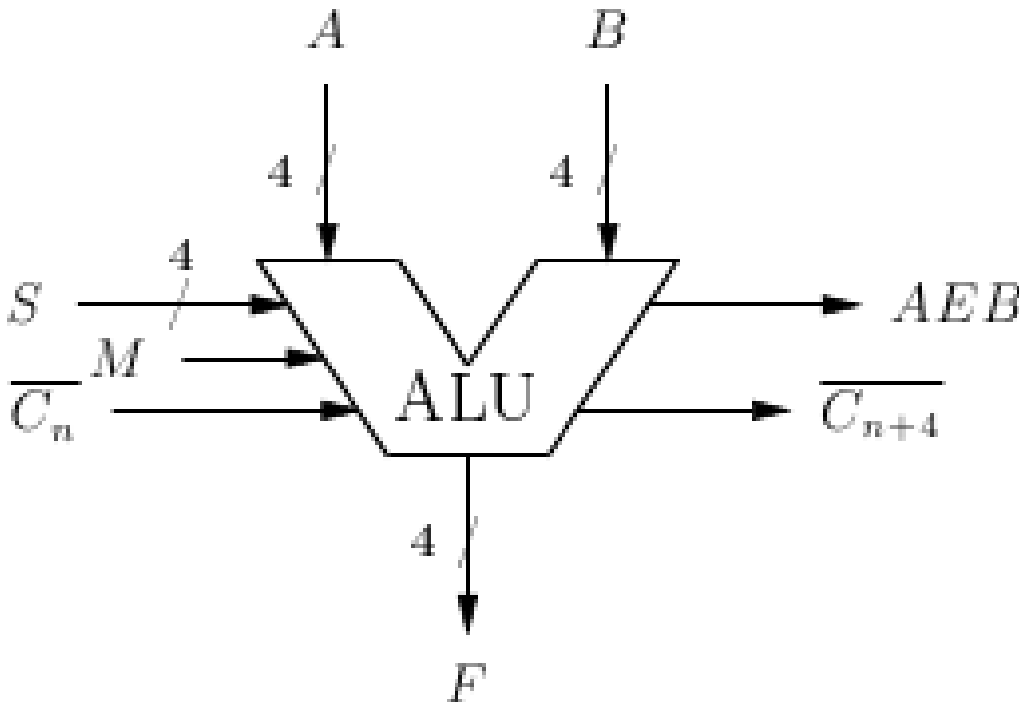


Figura10.1: Esquema funcional de la ALU 74181.

La Fig. 10.2 sirve como referencia para: tabla de operaciones de la ALU 74181.

S	Aritmético ($M = 0$)	Lógico ($M = 1$)
0000	$A + C_n$	$\overline{A}$
0001	$A \downarrow B + C_n$	$\overline{A \downarrow B}$
0010	$A \downarrow \overline{B} + C_n$	$\overline{A}B$
0011	$-1 + C_n$	0
0100	$A + A\overline{B} + C_n$	$\overline{A}B$
0101	$(A \downarrow B) + A\overline{B} + C_n$	$\overline{B}$
0110	$A - B - 1 + C_n$	$A \oplus B$
0111	$A\overline{B} - 1 + C_n$	$A\overline{B}$
1000	$A + AB + C_n$	$\overline{A} \downarrow B$
1001	$A + B + C_n$	$\overline{A \oplus B}$
1010	$(A \downarrow \overline{B}) + AB + C_n$	B
1011	$AB - 1 + C_n$	AB
1100	$A + A + C_n$	1
1101	$(A \downarrow B) + A + C_n$	$A \downarrow \overline{B}$
1110	$(A \downarrow \overline{B}) + A + C_n$	$A \downarrow B$
1111	$A - 1 + C_n$	A

Figura10.2: Tabla de operaciones de la ALU 74181.

10.5 Cierre

Antes de pasar a la sesión siguiente, guarda el fichero Verilog de cada ejercicio y anota dos cosas: que esperabas obtener y que imprimió realmente la simulación. Esa comparación es la forma más rápida de localizar errores.

10.6 Fuente original

Contenido redactado y ampliado a partir de la presentación de clase y de la página de referencia: <http://avellano.fis.usal.es/~compi/sesion9.htm>.